

Python: Lambda & Generators — Study Notes

Concise, beginner-friendly reference with definitions, roles, applications, when to use, and real-life examples.

Lambda Functions (Special-purpose / Anonymous)

A Definition

A *lambda* is a small anonymous function written on one line using the keyword `lambda`. It returns the value of a single expression.

```
# Syntax
# lambda arguments: expression

# Example
square = lambda x: x * x
print(square(5)) # 25
```

B Role

Create short, throwaway functions without the ceremony of `def`. Ideal for quick transformations and callback-style use.

C Application

Often used with built-ins such as `map()`, `filter()`, `sorted()`, and with event callbacks or GUI handlers where a tiny function is needed inline.

```
# map example
numbers = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, numbers))
print(squared) # [1, 4, 9, 16]

# sorted with key example
pairs = [(1, 'b'), (2, 'a'), (3, 'c')]
sorted_by_letter = sorted(pairs, key=lambda p: p[1])
```

D When to use

Use lambdas when the function is simple and used temporarily or inline. Prefer `def` for longer logic, multiple statements, or when you need a descriptive function name for readability.

E Real-life example

Adding 18% tax (VAT) to a list of prices:

```
prices = [100, 200, 300]
with_tax = list(map(lambda p: p + p * 0.18, prices))
print(with_tax) # [118.0, 236.0, 354.0]
```

Generators

A Definition

A *generator* is a function that yields items one at a time using the `yield` keyword instead of

returning them all at once. Calling a generator function returns an iterator that produces values on demand.

```
def count_up_to(n):
    x = 1
    while x <= n:
        yield x
        x += 1

# usage
for i in count_up_to(5):
    print(i) # prints 1,2,3,4,5 one per line
```

B Role

Produce sequences lazily (on demand). Generators are memory-efficient and perfect for iterating large or infinite sequences without loading everything into memory.

C Application

Use generators when reading large files line-by-line, streaming data, or producing large sequences like ranges, Fibonacci numbers, or database query results one at a time.

```
# Reading a large file line-by-line

def read_lines(file_name):
    with open(file_name) as f:
        for line in f:
            yield line.strip()

for ln in read_lines('big_log.txt'):
    # process each line without loading whole file
    print(ln)
```

D When to use

When dataset size is large, memory is limited, or when you want to stream or lazily compute values. Generators are also useful when you want to pause/resume function execution while maintaining local state.

E Real-life example

Processing a huge log file or dataset: you only keep one line/record in memory at a time, which prevents memory exhaustion and improves performance.

Quick Comparison

Concept	What it is	Best for
Lambda	Anonymous single-expression function	Short, inline transformations (map/filter/sorted)
Generator	Function that yields values one at a time	Large data, streaming, memory-efficient iteration